

Technologie Sieciowe

Warstwa transportu

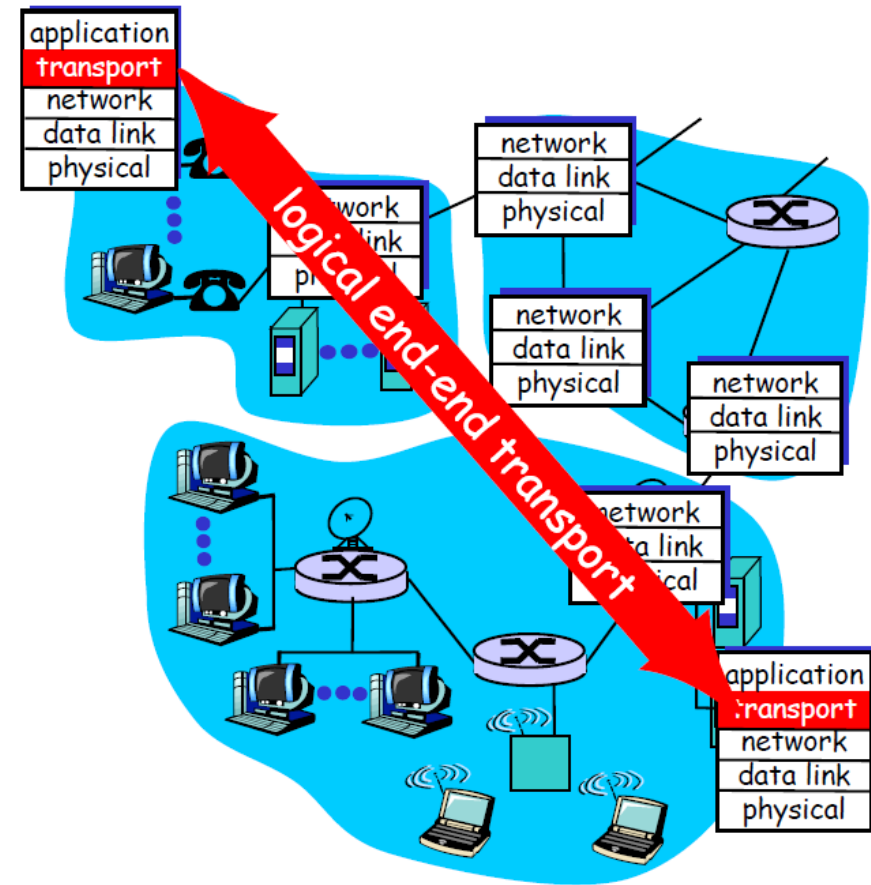
dr inż. Krzysztof Kosmowski

Mapa wykładu

- Usługi warstwy transportu
 - Multipleksacja i demultipleksacja
 - Transport bezpołączeniowy: UDP
 - Transport połączeniowy: TCP
 - struktura segmentu
 - niezawodna komunikacja
 - kontrola przepływu
 - zarządzanie połączeniem
 - Mechanizmy kontroli przeciążenia
 - Kontrola przeciążenia w TCP

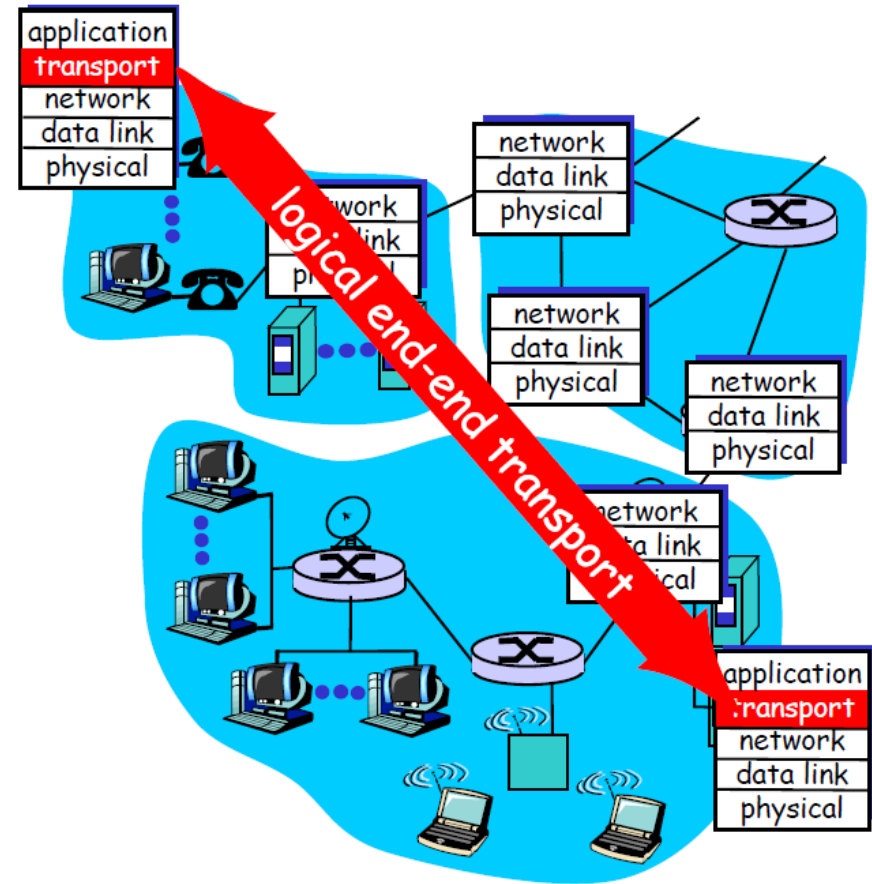
Usługi i protokoły warstwy transportu

- *logiczna komunikacja* pomiędzy procesami aplikacji działającymi na różnych hostach
- protokoły transportowe działają na systemach końcowych
 - **nadawca**: dzieli komunikat aplikacji na **segmenty**, przekazuje segmenty do warstwy sieci
 - **odbiorca**: łączy segmenty w komunikat, który przekazuje do warstwy aplikacji
- więcej niż jeden protokół transportowy
 - Internet: TCP oraz UDP



Warstwy transportu i sieci

- *warstwa sieci*: logiczna komunikacja pomiędzy hostami
- *warstwa transportu*: logiczna komunikacja pomiędzy procesami
 - korzysta z oraz uzupełnia usługi warstwy sieci



Protokoły transportowe Internetu

- niezawodna, uporządkowana komunikacja (TCP)
 - kontrola przeciążenia
 - kontrola przepływu
 - tworzenie połączenia
- zawodna, nieuporządkowana komunikacja (UDP)
 - proste rozszerzenie usługi “best-effort” IP
 - niedostępne usługi:
 - gwarancje maksymalnego opóźnienia
 - gwarancje minimalnej przepustowości

Mapa wykładu

- Usługi warstwy transportu
- Multipleksacja i demultipleksacja
- Transport bezpołączeniowy: UDP
- Transport połączeniowy: TCP
 - struktura segmentu
 - niezawodna komunikacja
 - kontrola przepływu
 - zarządzanie połączeniem
- Mechanizmy kontroli przeciążenia
- Kontrola przeciążenia w TCP

Multipleksacja/demultipleksacja

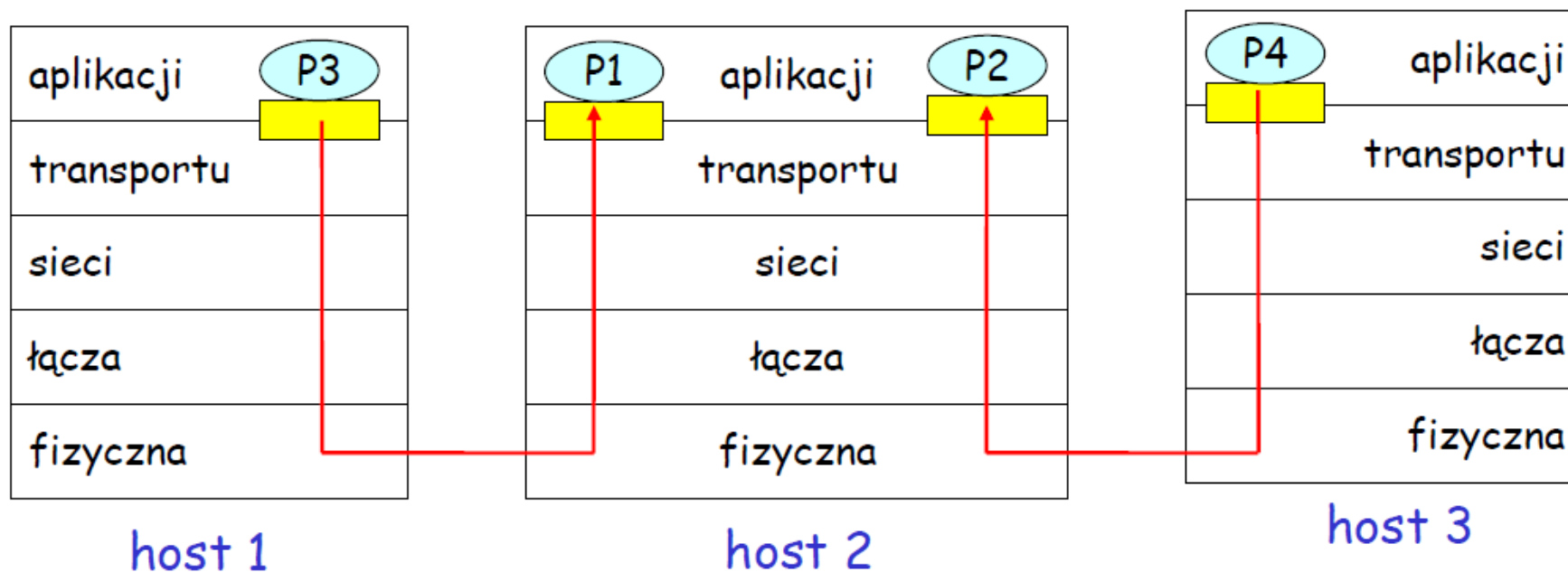
Demultipleksacja u odbiorcy

przekazywanie otrzymanych segmentów do właściwych gniazd

Multipleksacja u nadawcy

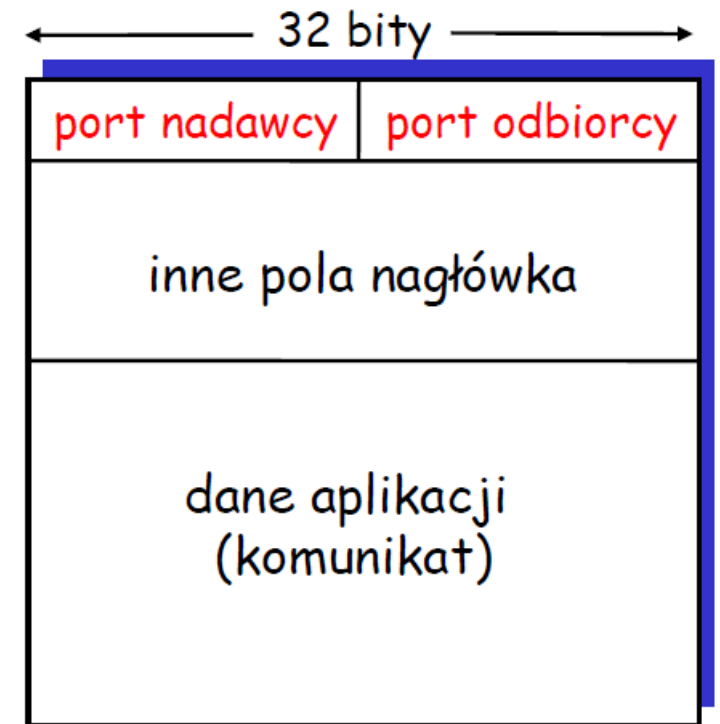
zbieranie danych z wielu gniazd, dodanie nagłówka (używanego później przy demultipleksacji)

 = gniazdo  = proces



Jak działa demultipleksacja

- **host otrzymuje pakiety IP**
- każdy pakiet ma adres IP nadawcy, adres IP odbiorcy
- każdy pakiet zawiera jeden segment warstwy transportu
- każdy segment ma port nadawcy i odbiorcy (pamiętać: powszechnie znane numery portów dla określonych aplikacji)
- **host używa adresu IP i portu żeby skierować segment do odpowiedniego gniazda**



format segmentu TCP/UDP

Demultipleksacja bezpołączeniowa

- Gniazda są tworzone przez podanie numeru portu:

```
DatagramSocket mojeGniazdo1 =  
new DatagramSocket(99111);  
DatagramSocket mojeGniazdo2 =  
new DatagramSocket(99222);
```

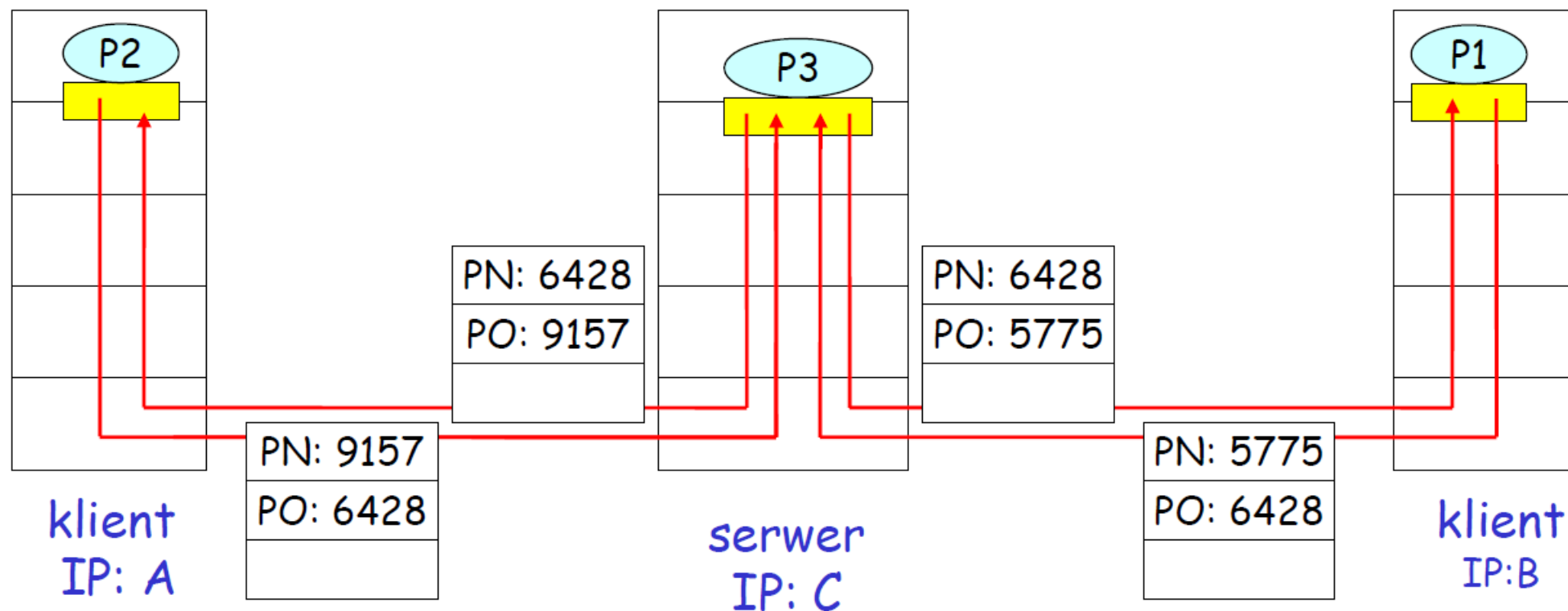
- Gniazdo UDP jest identyfikowane przez parę:

(adres IP odbiorcy, port odbiorcy)

- Kiedy host otrzymuje segment UDP:
 - sprawdza port odbiorcy w segmencie
 - kieruje segment UDP do gniazda z odpowiednim numerem portu
- Datagramy IP z różnymi adresami IP lub portami *nadawcy* są kierowane do tego samego gniazda

Demultipleksacja bezpołączeniowa (c.d.)

```
DatagramSocket gniazdoSerwera = new DatagramSocket(6428);
```

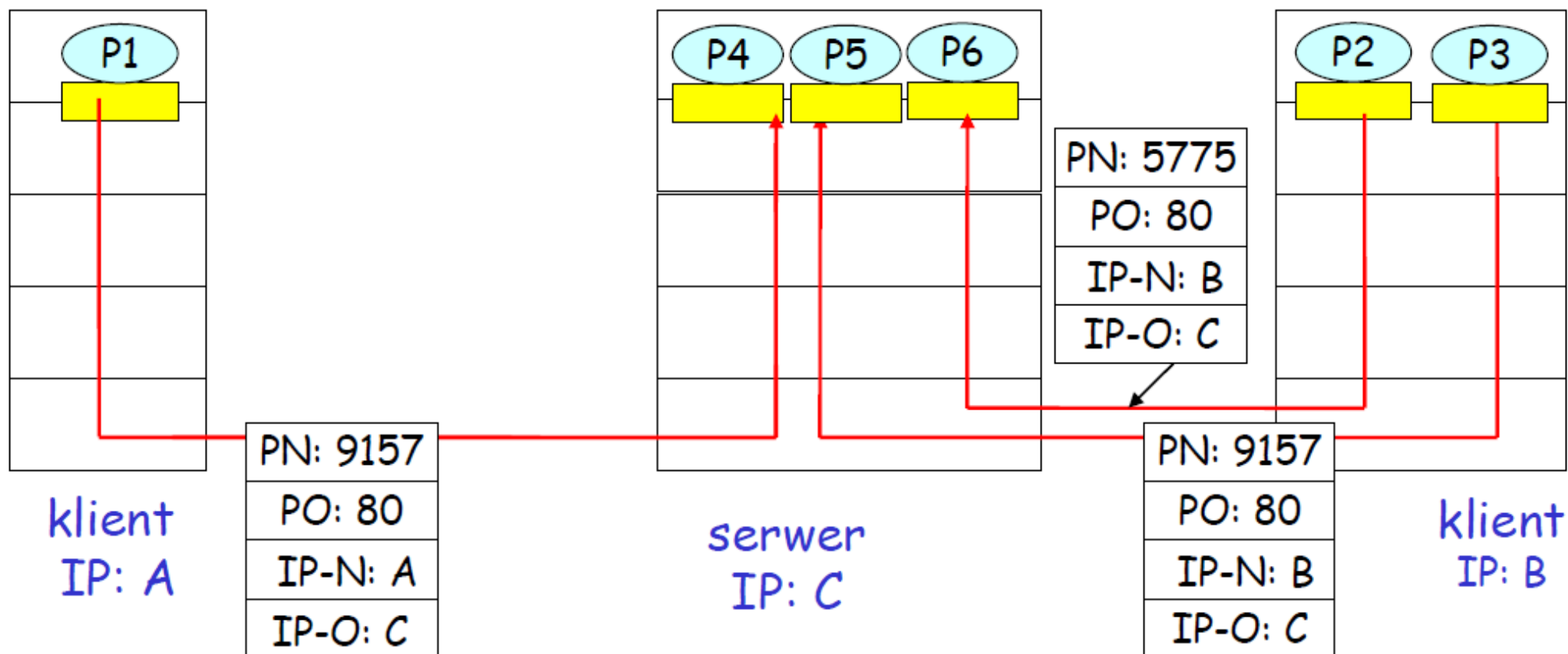


Port nadawcy (PN) jest „adresem zwrotnym”.

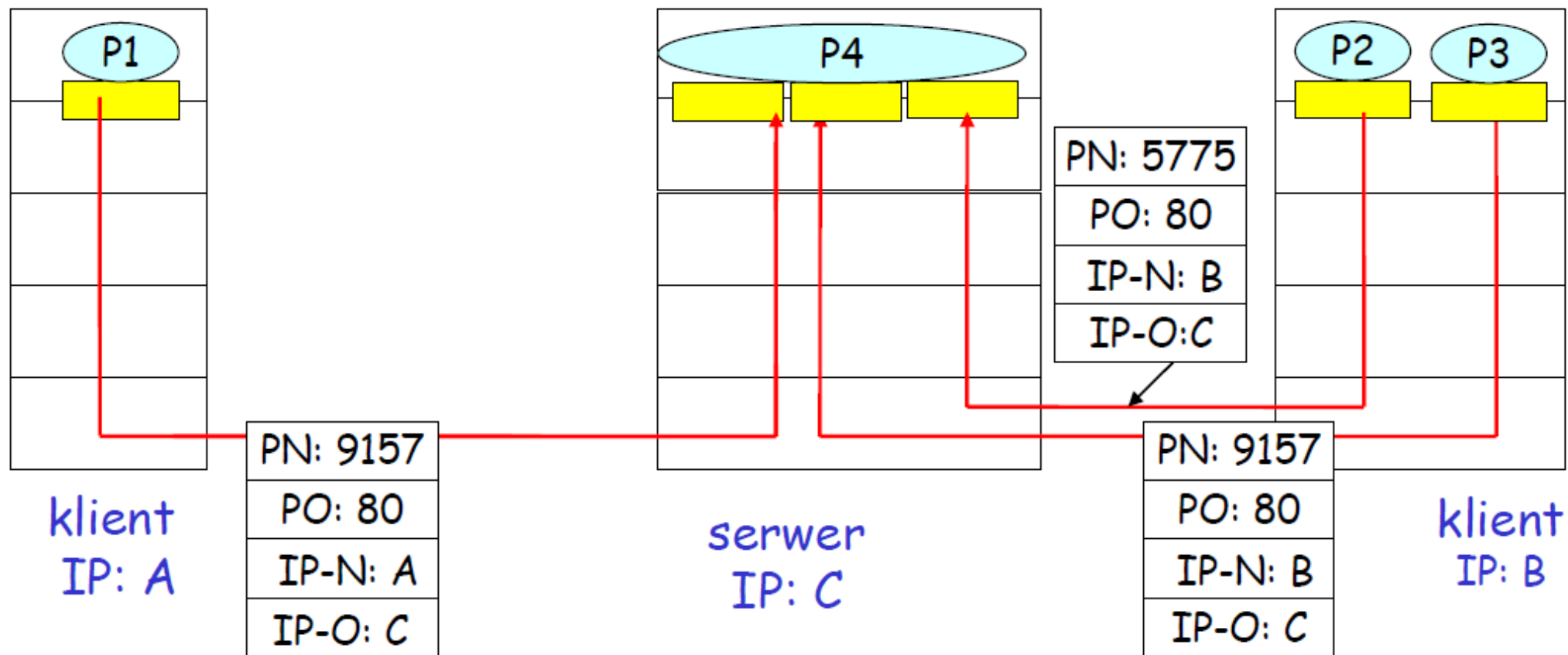
Demultipleksacja połączeniowa

- Gniazdo TCP jest określone przez cztery wartości:
 - adres IP nadawcy
 - port nadawcy
 - adres IP odbiorcy
 - port odbiorcy
- Host odbierający używa wszystkich 4 wartości, żeby skierować segment do właściwego gniazda
- Uwaga: host sprawdza także 5 wartość: **protokół**
- Host serwera może obsługiwać wiele gniazd TCP jednocześnie:
 - każde gniazdo ma inne 4 wartości
- Serwery WWW mają oddzielne gniazda dla każdego klienta
 - HTTP z nietrwałymi połączeniami wymaga oddzielnego gniazda dla każdego żądania

Demultipleksacja połączeniowa (c.d)



Demultiplexacja połączeniowa i serwer wielowątkowy



Mapa wykładu

- Usługi warstwy transportu
- Multipleksacja i demultipleksacja
- Transport bezpołączeniowy: UDP
- Transport połączeniowy: TCP
 - struktura segmentu
 - niezawodna komunikacja
 - kontrola przepływu
 - zarządzanie połączeniem
- Mechanizmy kontroli przeciążenia
- Kontrola przeciążenia w TCP

UDP: User Datagram Protocol [RFC 768]

- usługa typu “best effort”, segmenty UDP mogą zostać:
 - zgubione
 - dostarczone do aplikacji w zmienionej kolejności
- *bezpołączeniowy*:
 - nie ma inicjalizacji między nadawcą i odbiorcą UDP
 - każdy segment UDP jest obsługiwany niezależnie od innych

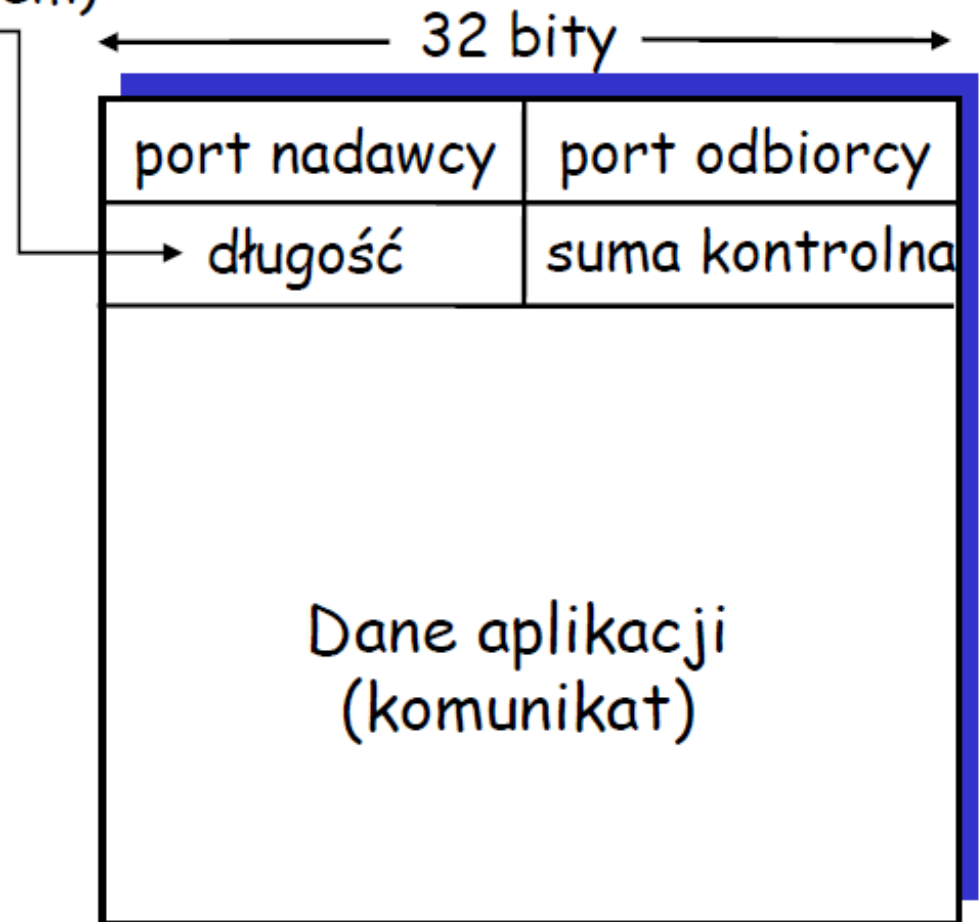
Czemu istnieje UDP?

- ❑ nie ma inicjalizacji połączenia (co może zwiększać opóźnienie)
- ❑ prosty: nie ma stanu połączenia u nadawcy ani odbiorcy
- ❑ mały nagłówek segmentu
- ❑ nie ma kontroli przeciążenia: UDP może śłać dane tak szybko, jak chce

Więcej o UDP

- Często używane do komunikacji strumieniowej
 - tolerującej straty
 - wrażliwej na opóźnienia
- Inne zastosowania UDP
 - DNS
 - SNMP
- niezawodna komunikacja po UDP: dodać niezawodność w warstwie aplikacji

Długość segmentu
UDP w bajtach
(z nagłówkiem)



Suma kontrolna UDP

Cel: odkrycie „błędów” (n.p., odwróconych bitów) w przesłanym segmencie

Nadawca:

- traktuje zawartość segmentu jako ciąg 16-bitowych liczb całkowitych
- suma kontrolna: dodawanie (i potem negacja sumy) zawartości segmentu
- nadawca wpisuje wartość sumy kontrolnej do odpowiedniego pola nagłówka UDP

Odbiorca:

- oblicza sumę kontrolną odebranego segmentu
- sprawdza, czy obliczona suma kontrolna jest równa tej, która jest w nagłówku:
 - NIE – wykryto błąd
 - TAK – Nie wykryto błędu.
- *Ale może błąd jest i tak?*

Wrócimy do tego

Przykład sumy kontrolnej

- Uwaga: Dodając liczby, reszta z dodawania najbardziej znaczących bitów musi zostać dodana do wyniku (zawinięta, przeniesiona na początek)
- Przykład: suma kontrolna dwóch liczb 16-bitowych

		1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
		1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
zawinięcie	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
suma		1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
suma kontrolna		0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

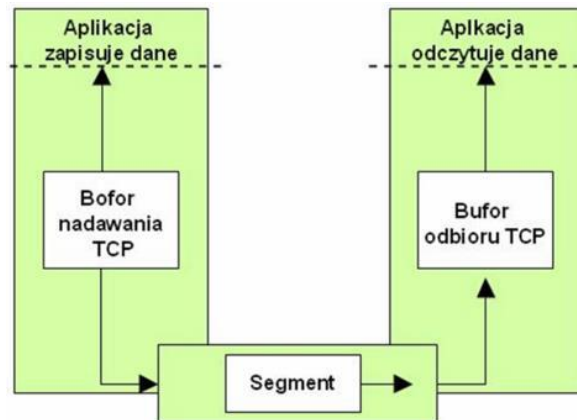
Mapa wykładu

- Usługi warstwy transportu
- Multipleksacja i demultipleksacja
- Transport bezpołączeniowy: UDP
- **Transport połączeniowy: TCP**
 - struktura segmentu
 - niezawodna komunikacja
 - kontrola przepływu
 - zarządzanie połączeniem
- Mechanizmy kontroli przeciążenia
- Kontrola przeciążenia w TCP

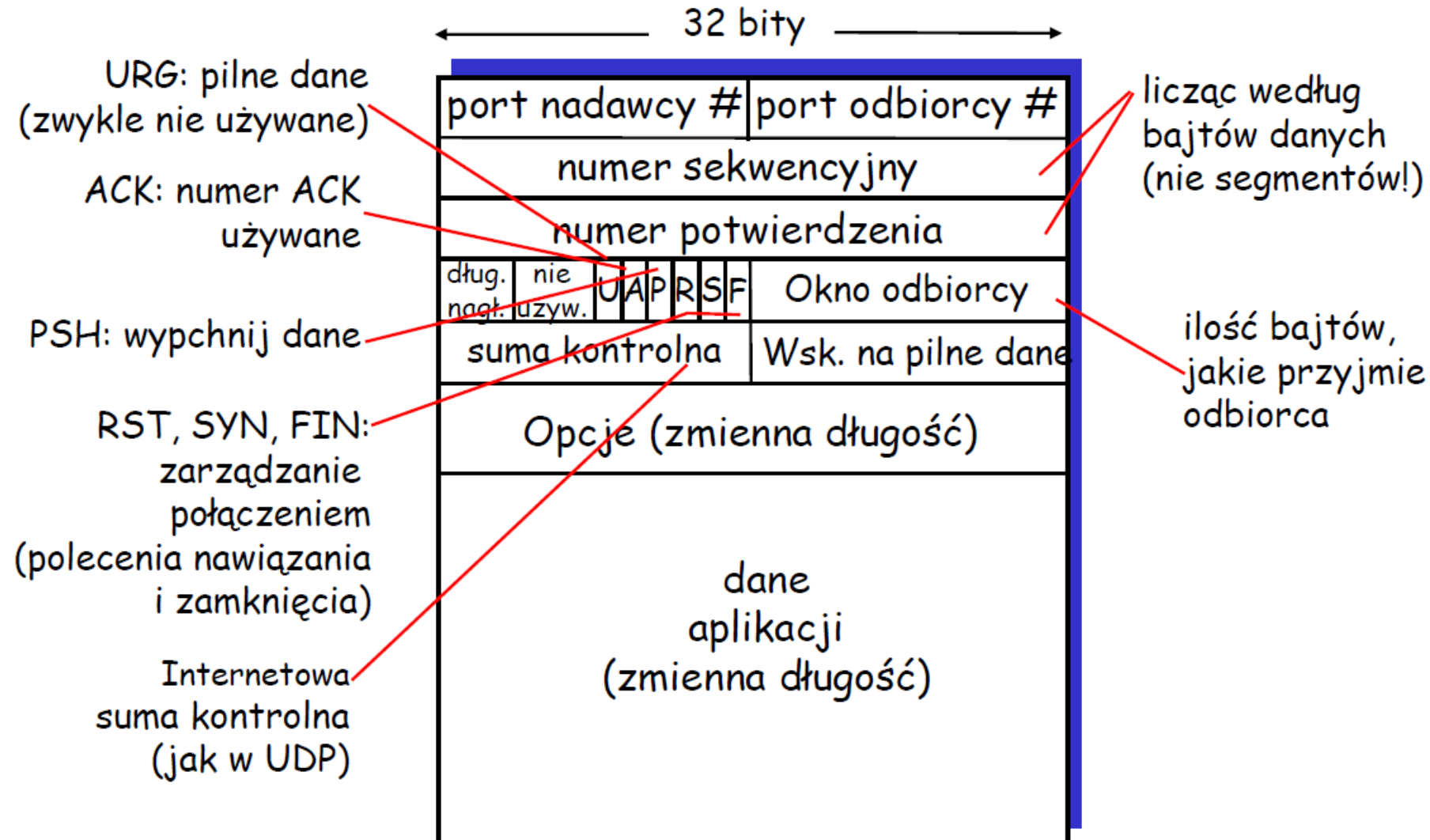
TCP: Przegląd

RFC: 793, 1122, 1323, 2018, 2581

- **koniec-koniec:**
 - jeden nadawca, jeden odbiorca
- **niezawodny, uporządkowany strumień bajtów:**
 - nie ma "granic komunikatów"
- **wysyłający grupowo:**
 - kontrola przeciążeń i przepływu TCP określają rozmiar okna
- **bufory u nadawcy i odbiorcy**
- **komunikacja "full duplex":**
 - dwukierunkowy przepływ danych na tym samym połączeniu
 - MRS: maksymalny rozmiar segmentu
- **połączeniowe:**
 - inicjalizacja (wymiana komunikatów kontrolnych) połączenia przed komunikacją danych
- **kontrola przepływu:**
 - nadawca nie "zaleje", odbiorcy



Struktura segmentu TCP



TCP: numery sekwencyjne i potwierdzenia

Numery sekwencyjne:

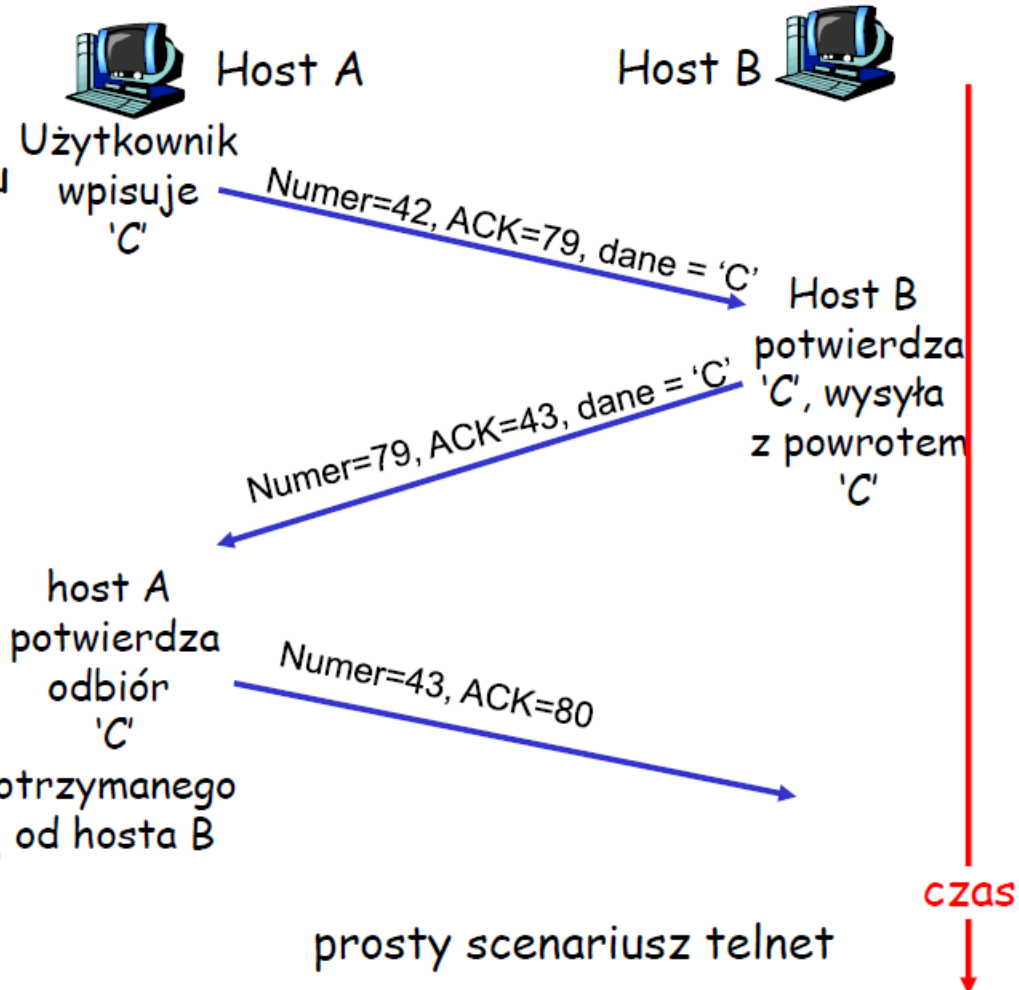
- numer w "strumieniu bajtów" pierwszego bajtu danych segmentu

Potwierdzenia:

- numer sekwencyjny następnego bajtu oczekiwanego od drugiej strony
- kumulatywny ACK

Pytanie: jak odbiorca traktuje segmenty nie w kolejności

- O: specyfikacja TCP tego nie określa: decyduje implementacja



TCP: czas powrotu (RTT) i timeout

Pytanie: jak ustalić timeout dla TCP?

- dłuższe niż RTT
 - ale RTT jest zmienne
- za krótki: za wczesny timeout
 - niepotrzebne retransmisje
- za długi: wolna reakcja na stratę segmentu

Pytanie: jak estymować RTT?

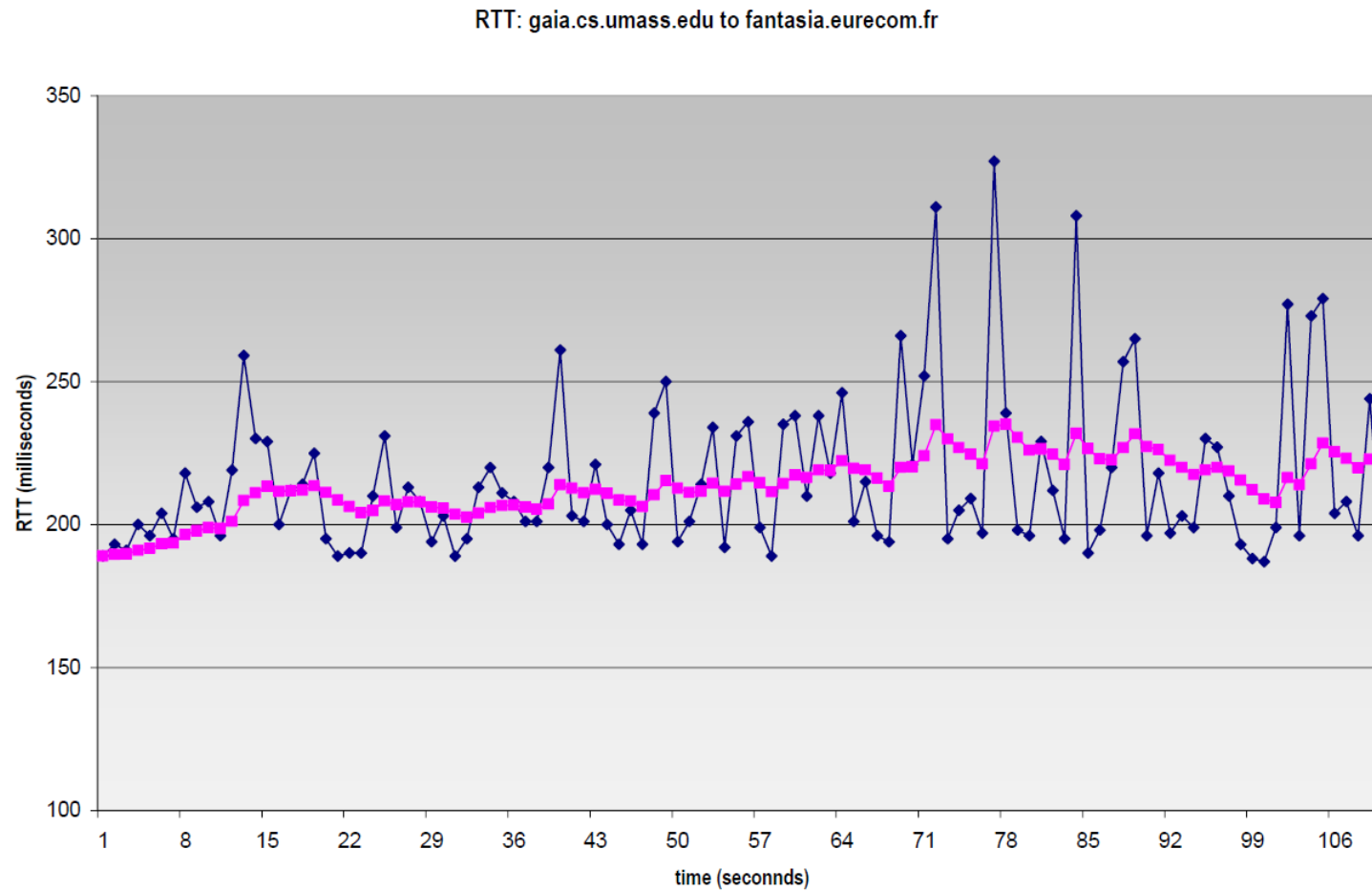
- **MierzoneRTT**: czas zmierzony od wysłania segmentu do odbioru ACK
 - ignorujemy retransmisje
- **MierzoneRTT** będzie zmienne, chcemy mieć "gładsze,, estymowane RTT
 - średnia z wielu ostatnich pomiarów, nie tylko aktualnego **MierzoneRTT**

TCP: czas powrotu (RTT) i timeout

$$\text{EstymowaneRTT} = (1 - \alpha) * \text{EstymowaneRTT} + \alpha * \text{MierzoneRTT}$$

- Wykładnicza średnia ruchoma
- wpływ starych pomiarów maleje wykładniczo
- typowa wartość parametru: $\alpha = 0.125$

Przykład estymacji RTT:



TCP: czas powrotu (RTT) i timeout

Ustawianie timeout

- **EstymowaneRTT** dodać “margines błędu”
- im większa zmienność **MierzoneRTT**, **tym** większy margines błędu
- najpierw ocenić, o ile **MierzoneRTT** jest oddalone od **EstymowaneRTT**:

$$\text{BładRTT} = (1-\beta) * \text{BładRTT} + \beta * |\text{MierzoneRTT} - \text{EstymowaneRTT}|$$

(zwykle, $\beta = 0.25$)

Ustalanie wielkości timeout

$$\text{Timeout} = \text{EstymowaneRTT} + 4 * \text{BładRTT}$$

Mapa wykładu

- Usługi warstwy transportu
- Multipleksacja i demultipleksacja
- Transport bezpołączeniowy: UDP
- Transport połączeniowy: TCP
 - struktura segmentu
 - niezawodna komunikacja
 - kontrola przepływu
 - zarządzanie połączeniem
- Mechanizmy kontroli przeciążenia
- Kontrola przeciążenia w TCP

Niezawodna komunikacja TCP

- TCP tworzy usługę NPK na zawodnej komunikacji IP
- Wysyłanie grupowe segmentów
- Kumulowane potwierdzenia
- Retransmisje są powodowane przez:
 - zdarzenia timeout
 - duplikaty ACK
- Na razie rozważymy prostszego nadawcę TCP:
 - ignoruje duplikaty ACK
 - ignoruje kontrolę przeciążenia i przepływu

Zdarzenia nadawcy TCP:

Dane od aplikacji:

- Utwórz segment z numerem sekwencyjnym
- numer sekwencyjny to numer w strumieniu bajtów pierwszego bajtu danych segmentu
- włącz zegar, jeśli jest wyłączony (zegar działa dla najstarszego niepotwierdzonego segmentu)
- oblicz czas oczekiwania:

Timeout

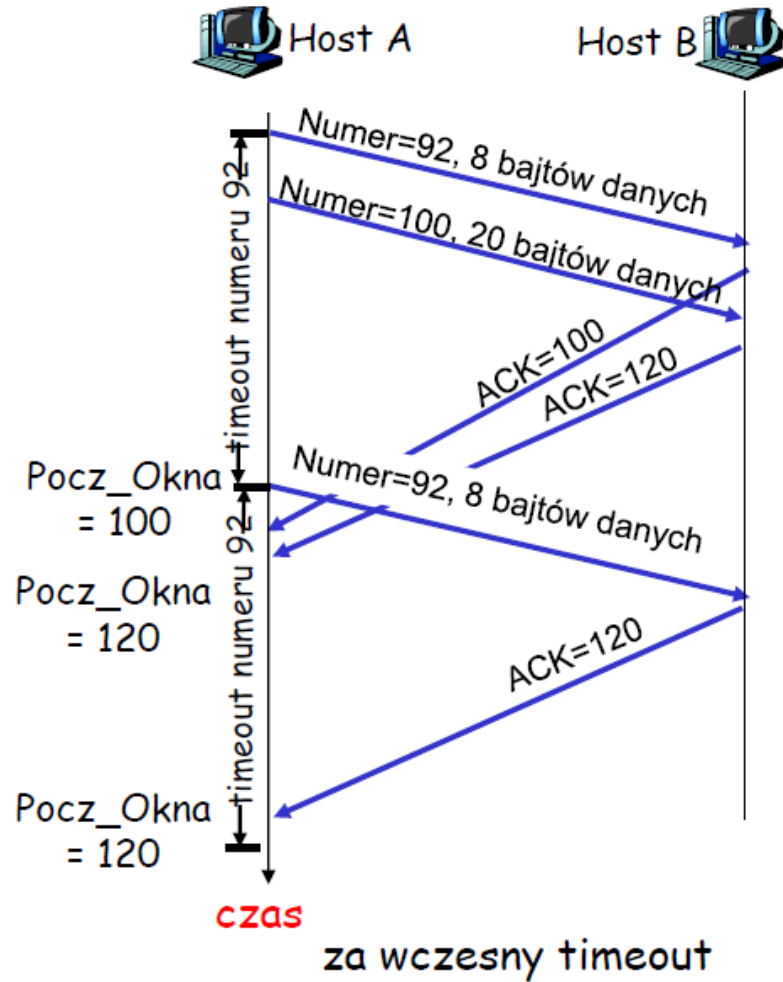
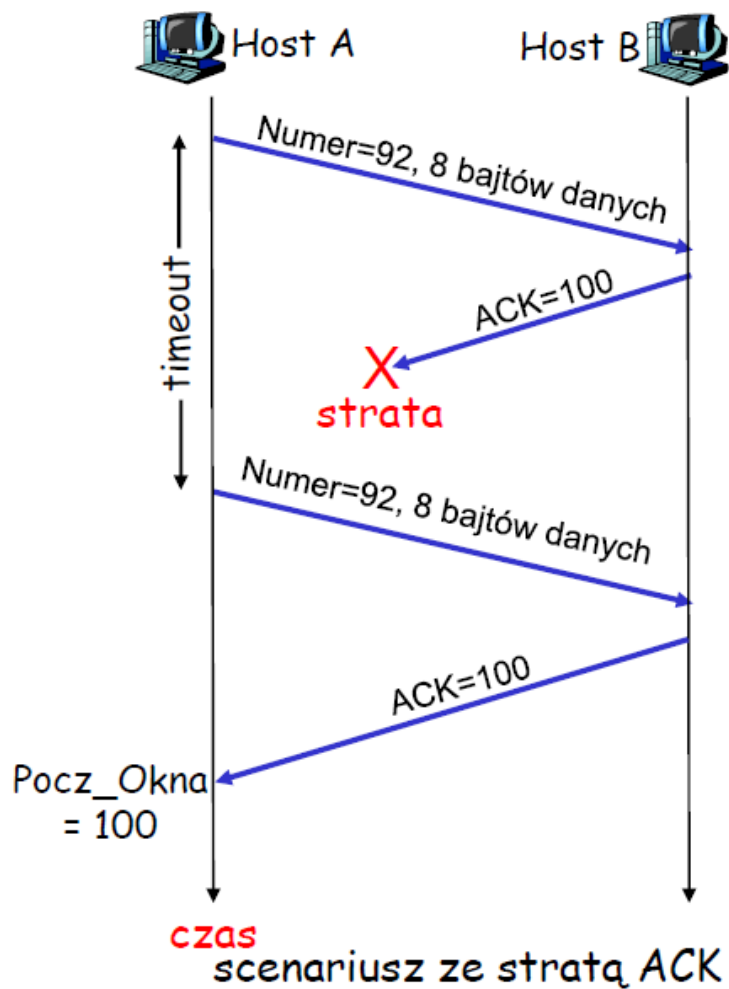
Timeout:

- retransmituj segment, który spowodował timeout
- ponownie włącz zegar

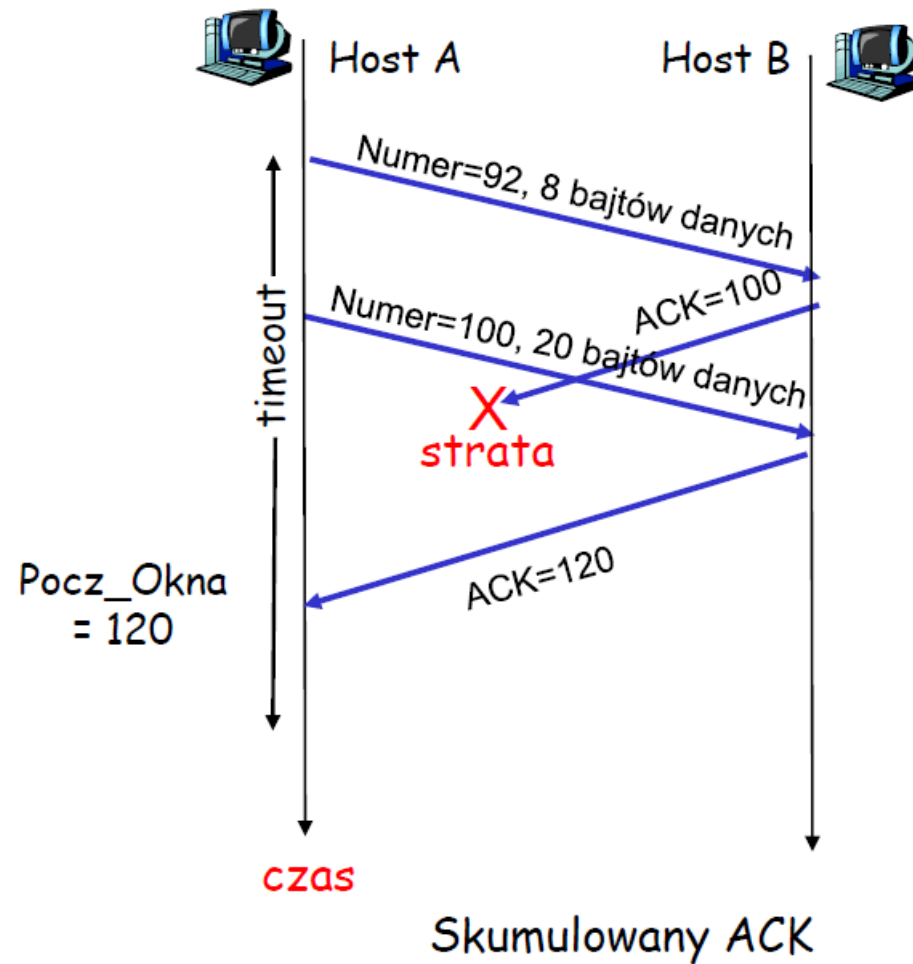
Odbiór ACK:

- Jeśli potwierdza niepotwierdzone segmenty:
 - potwierdź odpowiednie segmenty
 - włącz zegar, jeśli są brakujące segmenty

TCP: scenariusze retransmisji



TCP: scenariusze retransmisji (c.d.)



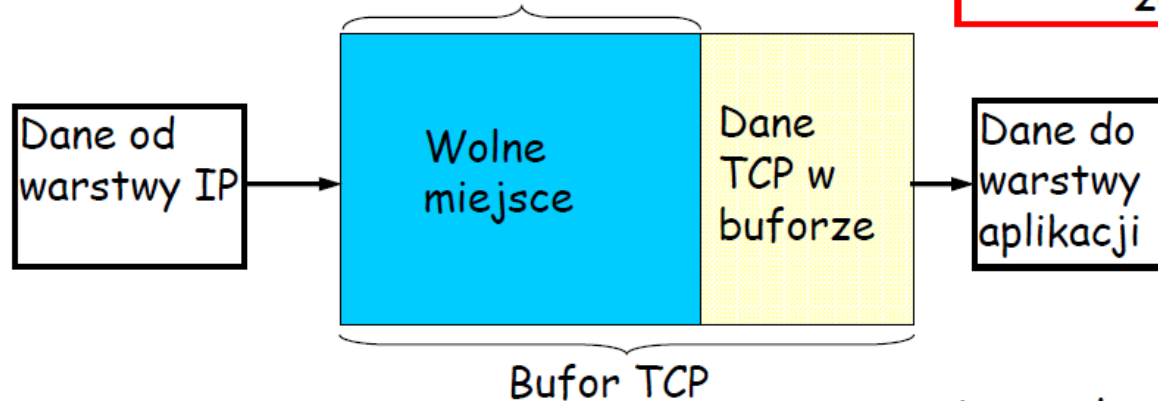
Mapa wykładu

- Usługi warstwy transportu
- Multipleksacja i demultipleksacja
- Transport bezpołączeniowy: UDP
- Transport połączeniowy: TCP
 - struktura segmentu
 - niezawodna komunikacja
 - kontrola przepływu
 - zarządzanie połączeniem
- Mechanizmy kontroli przeciążenia
- Kontrola przeciążenia w TCP

Kontrola przepływu TCP

- ❑ odbiorca TCP ma bufor odbiorcy:

Okno odbiorcy TCP



kontrola przepływu

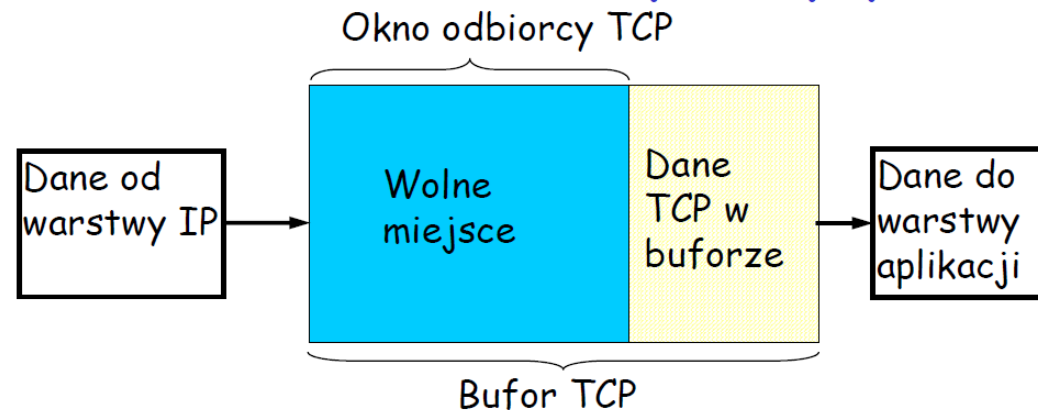
nadawca nie "zaleje"
bufora odbiorcy
przesyłając za dużo i
za szybko

- ❑ Proces aplikacji może za wolno czytać z bufora
- ❑ usługa dopasowania prędkości: dopasuje prędkość wysyłania do prędkości odbioru danych przez aplikację

Jak działa kontrola przepływu TCP

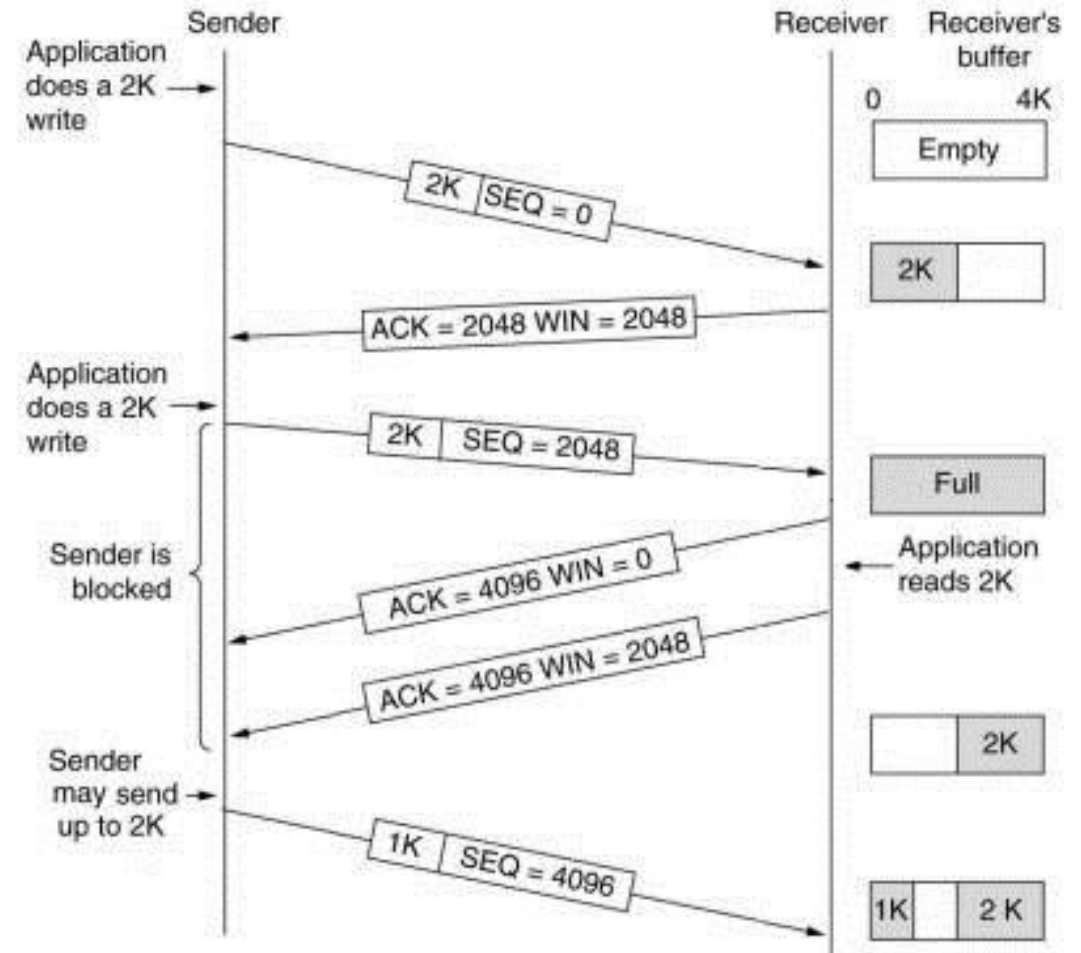
Założmy, że odbiorca wyrzuca segmenty nie w kolejności.

- Odbiorca ogłasza wolne miejsce umieszczając jego wielkość w segmentach ACK
- Odbiorca ogranicza rozmiar okna do wolnego miejsca
- gwarantuje, że bufor nie zostanie przepełniony



TCP – mechanizm przesuwającego okna

- Przesyłanie/potwierdzanie pojedynczych segmentów – Nieefektywne !
- Mechanizm przesuwającego okna (*ang. sliding window*)
- **Ustalenie rozmiaru okna** (na przykładzie po prawej – max 4k)
- Wysłanie danych:
 - W blokach
 - Po potwierdzeniu odebrania poprzedniego segmentu
- **Mechanizm kontroli przepływu**
 - Zmienny rozmiar okna
 - możliwość zablokowania nadawcy



Mapa wykładu

- Usługi warstwy transportu
- Multipleksacja i demultipleksacja
- Transport bezpołączeniowy: UDP
- Transport połączeniowy: TCP
 - struktura segmentu
 - niezawodna komunikacja
 - kontrola przepływu
 - zarządzanie połączeniem
- Mechanizmy kontroli przeciążenia
- Kontrola przeciążenia w TCP

Zarządzanie połączeniem TCP

Przypomnienie: Nadawca i odbiorca TCP, tworzą “połączenie” zanim wymienią dane

- inicjalizacja zmiennych TCP:
 - numery sekwencyjne
 - bufory, informacja kontroli przepływu (rozmiar bufora)
- klient: nawiązuje połączenie

```
Socket clientSocket = new  
Socket("hostname", "port number");
```

- serwer: odbiera połączenie

```
Socket connectionSocket =  
welcomeSocket.accept();
```

Trzykrotny uścisk dłoni:

- **Krok 1:** host klienta wysyła segment SYN do serwera
 - podaje początkowy numer sekwencyjny
 - bez danych
- **Krok 2:** host serwera odbiera SYN, odpowiada segmentem SYNACK
 - serwer alokuje bufory
 - określa początkowy numer
- **Krok 3:** klient odbiera SYNACK, odpowiada segmentem ACK, który może zawierać dane

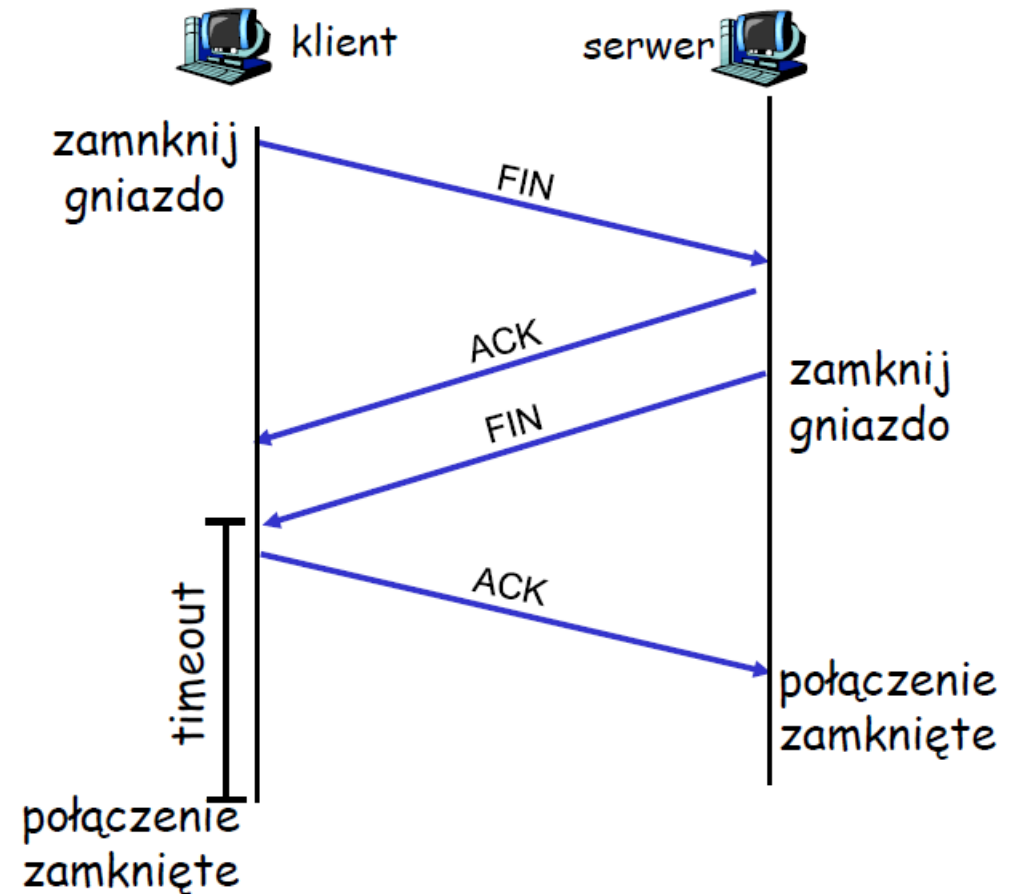
Zarządzanie połączeniem TCP (c.d..)

Zamykanie połączenia:

klient zamyka gniazdo:

```
clientSocket.close();
```

- **Krok 1:** Host **klienta** wysyła segment TCP FIN do serwera
- **Krok 2:** **serwer** odbiera FIN, odpowiada segmentem
- ACK. Zamyka połączenie, wysyła FIN



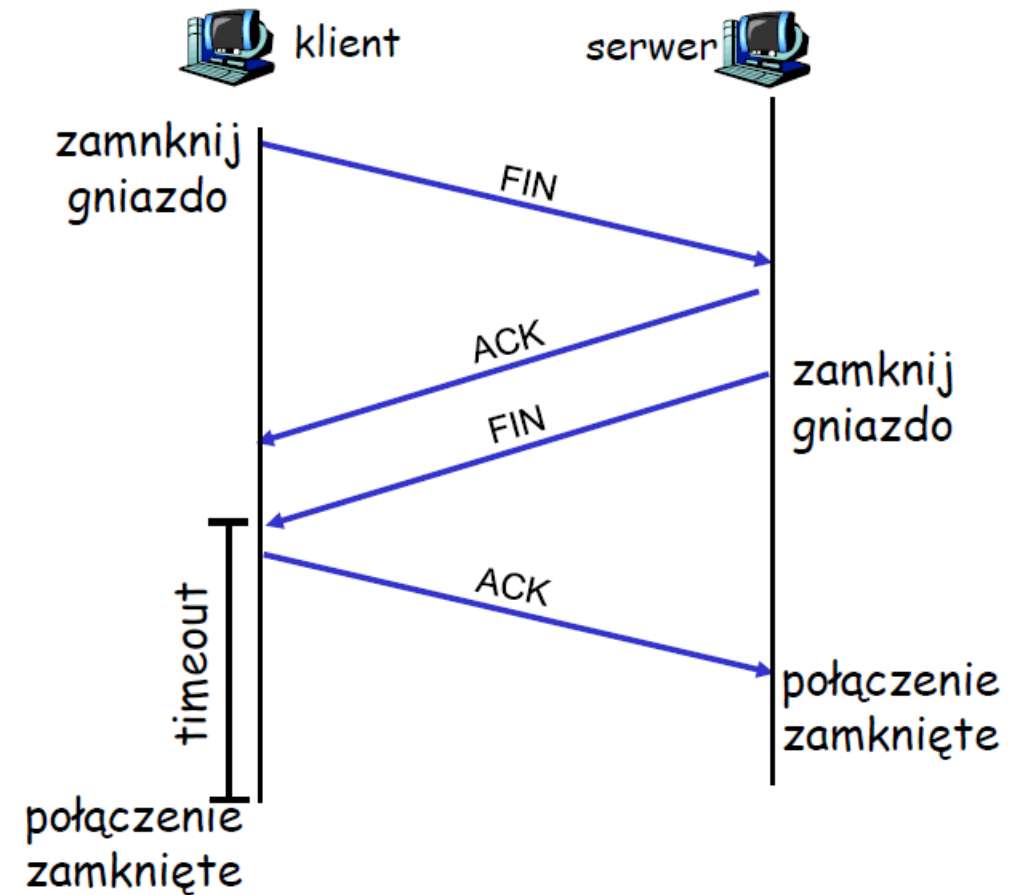
Zarządzanie połączeniem TCP (c.d.)

Krok 3: klient odbiera FIN, odpowiada segmentem ACK

- Rozpoczyna oczekiwanie – będzie odpowiadał ACK na otrzymane FIN.

Krok 4: serwer odbiera ACK. Połączenie zamknięte.

Uwaga: z małymi zmianami, obsługuje jednocześnie FIN



Mapa wykładu

- Usługi warstwy transportu
- Multipleksacja i demultipleksacja
- Transport bezpołączeniowy: UDP
- Transport połączeniowy: TCP
 - struktura segmentu
 - niezawodna komunikacja
 - kontrola przepływu
 - zarządzanie połączeniem
- Mechanizmy kontroli przeciążenia
- Kontrola przeciążenia w TCP

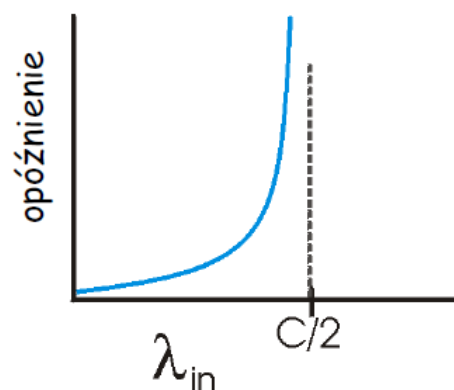
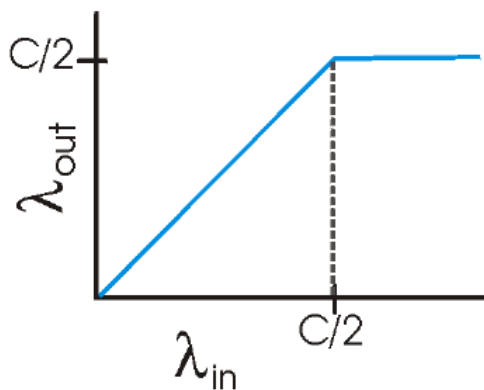
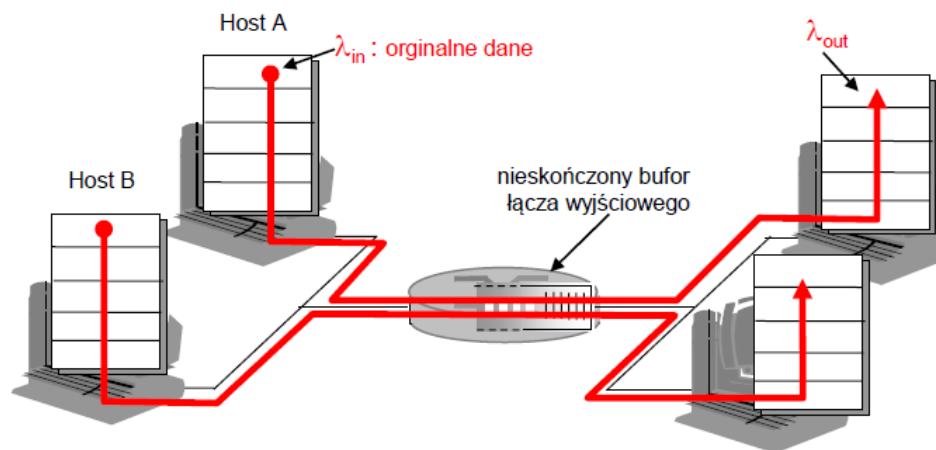
Zasady kontroli przeciążenia

Przeciążenie:

- nieformalnie: “za wiele źródeł wysyła za wiele danych za szybko, żeby *sieć* mogła je obsłużyć”
- różni się od kontroli przepływu!
- objawy przeciążenia:
 - zgubione pakiety (przepełnienie buforów w ruterach)
 - duże (i zmienne) opóźnienia (kolejkowanie w buforach ruterów)
- jeden z głównych problemów sieci!
- konsekwencja braku kontroli dostępu do sieci

Przyczyny/koszty przeciążenia: scenariusz 1

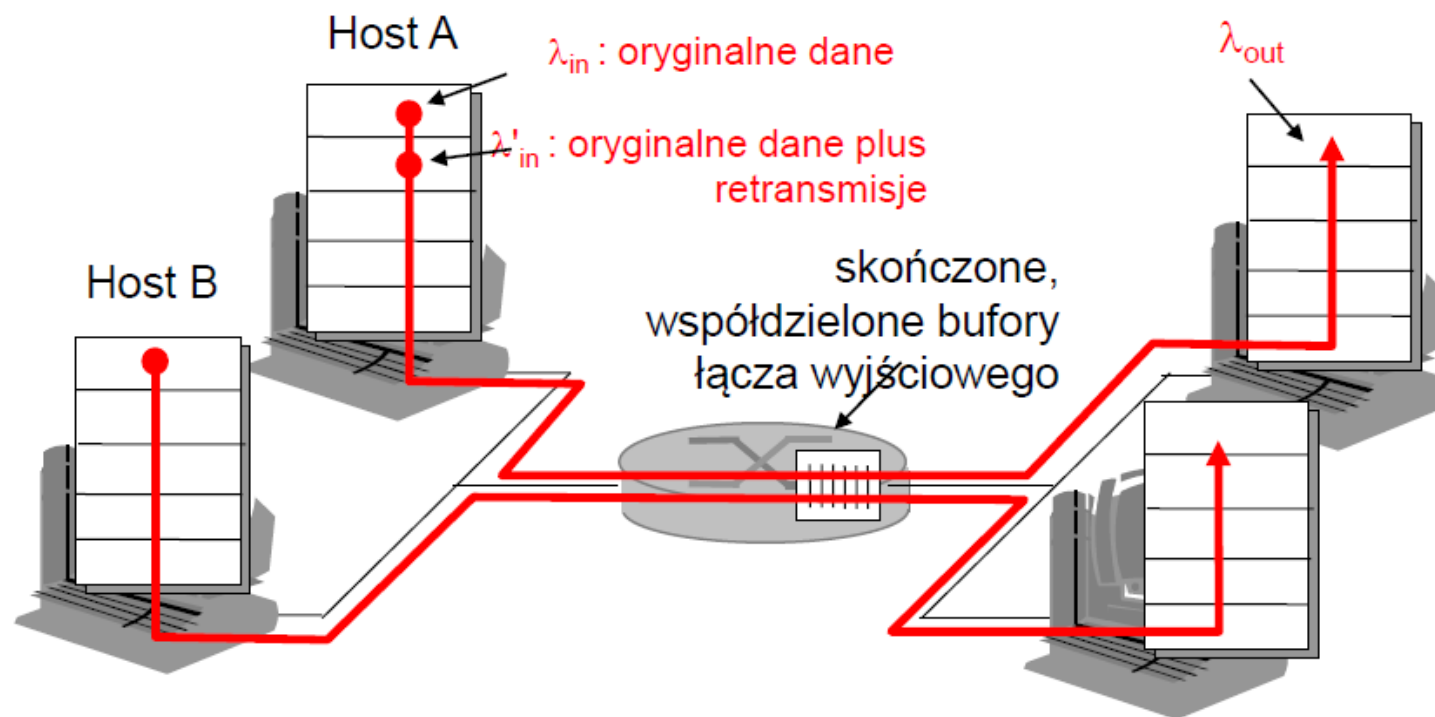
- dwóch nadawców, dwóch odbiorców
- jeden ruter, nieskończone bufony
- bez retransmisji



- duże opóźnienia przy przeciążeniu
- maksymalna dostępna przepustowość

Przyczyny/koszty przeciążenia: **scenariusz 2**

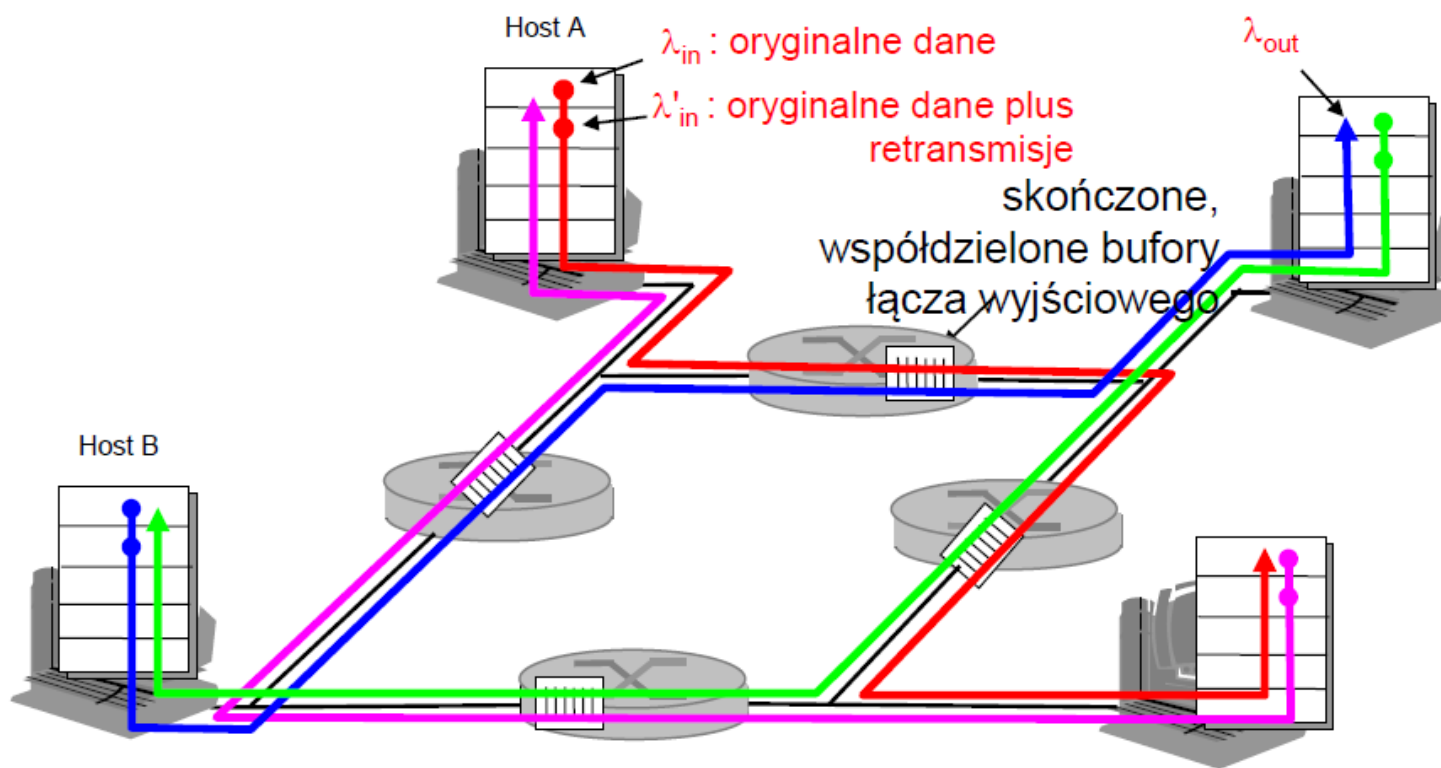
- ❑ jeden ruter, *skończone* bufony
- ❑ retransmisje straconych pakietów



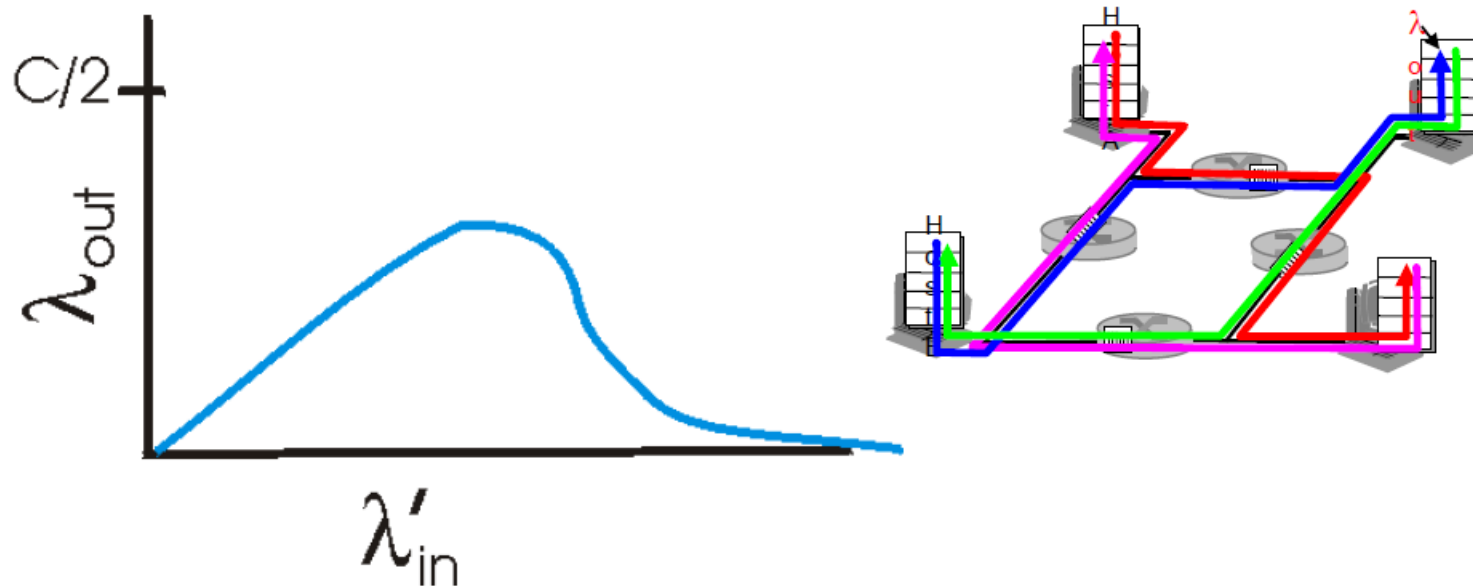
Przyczyny/koszty przeciążenia: scenariusz 3

- ❑ czterech nadawców
- ❑ dłuższe ścieżki
- ❑ timeout/retransmisje

Pytanie: co nastąpi,
gdy λ_{in} oraz λ'_{in}
wzrosną?



Przyczyny/koszty przeciążenia: scenariusz 3



Jeszcze jeden "koszt" przeciążenia:

- gdy pakiet ginie, przepustowość "w górę ścieżki" zużyta na ten pakiet została zmarnowana!

Metody kontroli przeciążenia

Dwa rodzaje metod kontroli przeciążenia:

Kontrola przeciążenia typu koniec-koniec:

- brak bezpośredniej informacji zwrotnej od warstwy sieci
- przeciążenie jest obserwowane na podstawie strat i opóźnień w systemach końcowych
- tą metodą posługuje się TCP

Kontrola przeciążenia z pomocą sieci:

- routery udostępniają informację zwrotną systemom końcowym
- pojedynczy bit wskazuje na przeciążenie (SNA, DECbit, TCP/IP ECN, ATM)
- podawana jest dokładna prędkość, z jaką system końcowy powinien wysłać

Studium przypadku: kontrola przeciążeń w usłudze ABR sieci ATM

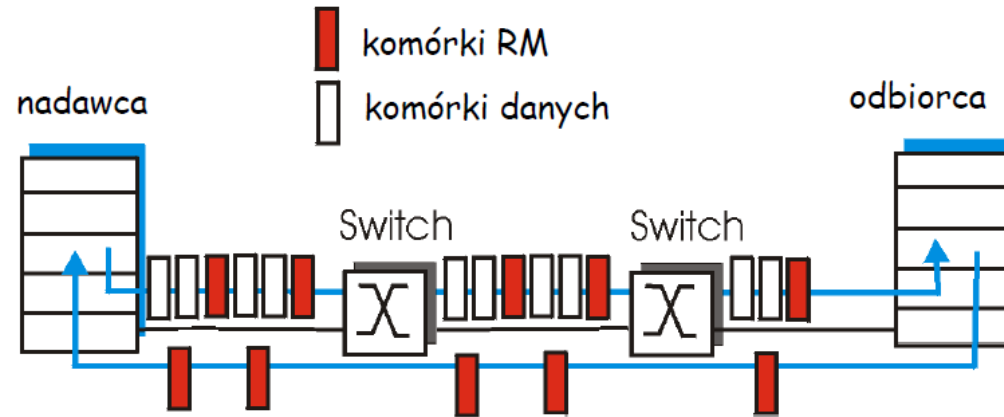
ABR: available bit rate:

- “usługa elastyczna”
- jeśli ścieżka nadawcy jest “niedociążona”:
 - nadawca powinien używać dostępnej przepustowości
- jeśli ścieżka nadawcy jest przeciążona:
 - nadawca jest ograniczany do minimalnej gwarantowanej przepustowości

Komórki RM (resource management):

- wysyłane przez nadawcę, przeplatane z komórkami danych
- bity w komórce RM ustawiane przez przełączniki sieci (“z pomocą sieci”)
 - bit NI: nie zwiększaj szybkości (lekkie przeciążenie)
 - bit CI: wskazuje na przeciążenie
- komórki RM zwracane są do nadawcy przez odbiorcę bez zmian

Studium przypadku: kontrola przeciążeń w usłudze ABR sieci ATM



- dwubajtowe pole ER (explicit rate) w komórce RM
 - przeciążony switch może zmniejszyć wartość ER w komórce
 - z tego powodu, nadawca ma minimalną dostępną przepustowość na ścieżce
- bit EFCI w komórkach danych: ustawiany na 1 przez przeciążony switch
 - jeśli komórka danych poprzedzająca komórkę RM ma ustawiony bit EFCI, odbiorca ustawia bit CI w zwróconej komórce RM

Mapa wykładu

- Usługi warstwy transportu
- Multipleksacja i demultipleksacja
- Transport bezpołączeniowy: UDP
- Transport połączeniowy: TCP
 - struktura segmentu
 - niezawodna komunikacja
 - kontrola przepływu
 - zarządzanie połączeniem
- Mechanizmy kontroli przeciążenia
- Kontrola przeciążenia w TCP

Kontrola przeciążenia w TCP

metoda koniec-koniec (bez pomocy sieci)

- ❑ nadawca ogranicza prędkość transmisji:

$$\frac{\text{OstatniWysłanyBajt} - \text{OstatniPotwierdzonyBajt}}{\text{RozmiarOkna}} \leq \text{prędkość}$$

- ❑ Z grubsza,

$$\text{prędkość} = \frac{\text{RozmiarOkna}}{\text{RTT}} [\text{Bajt/s}]$$

- ❑ RozmiarOkna jest zmienny, funkcja obserwowanego przeciążenia sieci

Jak nadawca obserwuje przeciążenie?

- ❑ strata = timeout lub 3 zduplikowane ACK
- ❑ nadawca TCP zmniejsza prędkość (RozmiarOkna) po stracie

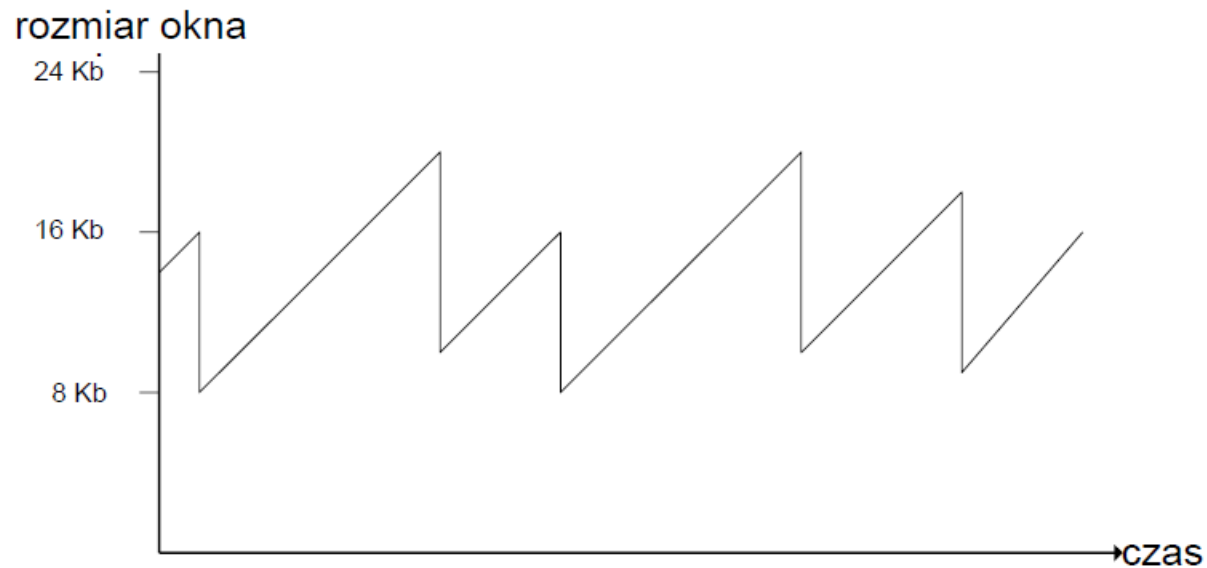
trzy mechanizmy:

- AIMD
- powolny start
- konserwatywne po zdarzeniu timeout

Mechanizm AIMD w TCP

Multiplikatywne zmniejszanie: dziel
RozmiarOkna przez dwa po stracie

addytywne zwiększanie: zwiększ
RozmiarOkna o 1 rozmiar segmentu
po każdym RTT bez straty: *badanie sieci*



Długotrwałe połączenie TCP

Powolny start TCP

- Po nawiązaniu połączenia,
RozmiarOkna = 1 segment
- Przykład: Segment = 500 b oraz RTT = 200 ms
- początkowa prędkość = 2.5 kb/s
- dostępna przepustowość >> rozmiarSegmentu/RTT
- pożądane jest szybkie przyśpieszenie komunikacji
- Na początku połączenia, rozmiar okna jest mnożony przez dwa aż do wystąpienia pierwszej straty
- Potem zaczyna działać mechanizm AIMD

□ Z grubsza,

$$\text{prędkość} = \frac{\text{RozmiarOkna}}{\text{RTT}} [\text{Bajt/s}]$$

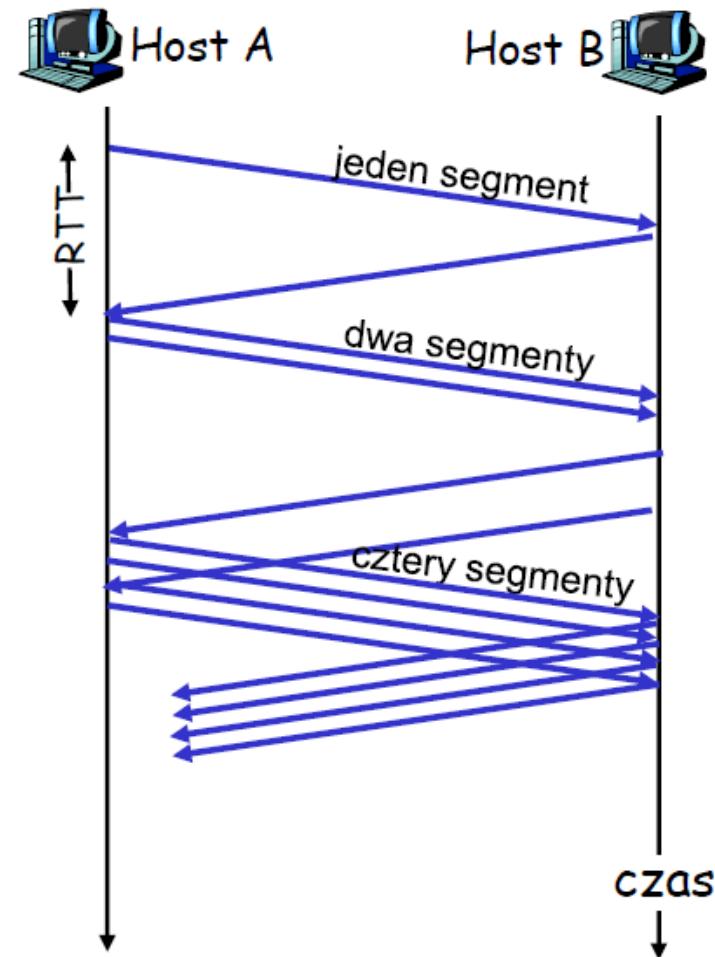
Powolny start TCP (c.d.)

Po nawiązaniu połączenia, zwiększaj prędkość wykładniczo do pierwszej straty:

- podwajaj **RozmiarOkna** co RTT
- osiągane przez zwiększanie **RozmiarOkna** po otrzymaniu ACK

Podsumowanie:

początkowo, prędkość jest mała, ale szybko przyspiesza

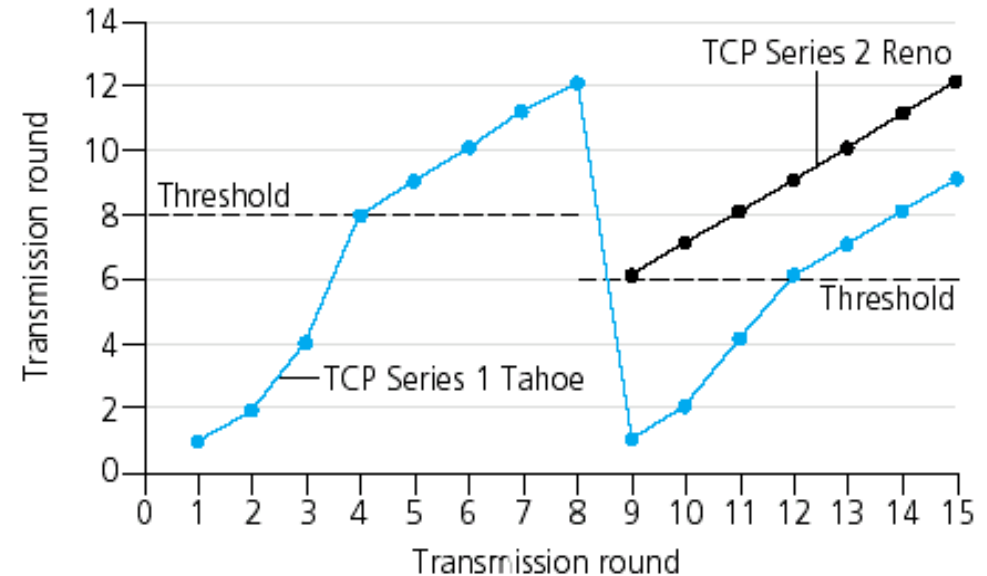


Udoskonalenie powolnego startu

- Po 3 zduplikowanych ACK:
 - **RozmiarOkna** dzielimy przez dwa
 - okno zwiększane liniowo
- Ale: po zdarzeniu timeout:
 - **RozmiarOkna** ustawiany na 1 segment;
 - okno z początku zwiększane wykładniczo
 - do pewnej wielkości, potem zwiększane liniowo

Udoskonalenia (c.d.)

- Pytanie: Kiedy przestać zwiększać okno wykładniczo, a zacząć zwiększać liniowo?
- Odpowiedź: Gdy **RozmiarOkna** osiągnie połowę swojej wartości z przed zdarzenia timeout



Implementacja:

- Zmienny *Próg*
- Po stracie, Próg jest ustalany na $1/2$ wartości RozmiarOkna tuż przed stratą

Podsumowanie: kontrola przeciążenia TCP

- Gdy **RozmiarOkna** poniżej wartości **Próg**, nadawca w stanie **powolnego startu**, okno rośnie wykładniczo.
- Gdy **RozmiarOkna** powyżej wartości **Próg**, nadawca w stanie **unikania przeciążenia**, okno rośnie liniowo
- Po **trzech zduplikowanych ACK**, $\text{Próg} = \text{RozmiarOkna} / 2$ oraz $\text{RozmiarOkna} = \text{Próg}$.
- Po zdarzeniu **timeout**, $\text{Próg} = \text{RozmiarOkna} / 2$ oraz $\text{RozmiarOkna} = 1$ segment.

Przepustowość TCP

- Jaka jest średnia przepustowość TCP jako funkcja rozmiaru okna oraz RTT?
 - Ignorujemy powolny start
- Niech **W** będzie rozmiarem okna w chwili straty.
- Gdy okno ma rozmiar **W** , przepustowość jest **W/RTT**
- Zaraz po stracie, okno zmniejszane do **$W/2$** , przepustowość jest **$W/2RTT$** .
- Średnia przepustowość: **$\frac{3}{4} W/RTT$**

Podsumowanie warstwy transportu

- mechanizmy usług warstwy transportu:
 - multipleksacja, demultipleksacja
 - niezawodna komunikacja
 - kontrola przepływu
 - kontrola przeciążenia
- w Internecie:
 - UDP
 - TCP
- Co dalej:
 - opuszczamy “brzeg” sieci (warstwy aplikacji i transportu)
 - wchodzimy w “szkielet” sieci

KONIEC